

Looking at your codebase, here's a precise agent prompt built around your actual file structure and eval pipeline:

SON OF ANTON — AGENT PROMPT

Identity

You are operating inside the Son of Anton codebase.
Owner: Mohammed Rehan (Maverick)
Goal: Recruiter-facing AI persona — grounded answers, scheduling, voice + chat.

MANDATORY READING ORDER (before ANY task)

- 1. son_of_anton_master_doc.md → architecture source of truth
- 2. AGENT.md → implementation rules
- 3. Current task instructions
- 4. Inspect affected files
- 5. Produce implementation plan
- 6. Only then write code

Skipping this order = violation. Never redesign the architecture.

ARCHITECTURE (non-negotiable)

Voice / Chat UI
↓
Anton Orchestrator (LangGraph)
↓
Knowledge Layer | Conversation Layer | Action Layer
↓
Context Assembly → LLM → Response

Retrieval flow: Cache → Vector Store → (GitHub fallback only if stale/missing)
Calendar is the source of truth. DB stores metadata only.
Router: Fast keyword first → LLM fallback only if confidence < 0.7

STACK (do not introduce new infra)

Backend: FastAPI + LangGraph + ChromaDB + Redis + PostgreSQL
LLM: GPT-4o / GPT-4o-mini
Voice: Vapi (Custom LLM mode)
Calendar: Google Calendar API
Observability: LangSmith

Frontend: Next.js + Tailwind

Hosting: Vercel (frontend) + Render (backend)

EVAL REPORT — PRIMARY OBJECTIVE

Run and capture all measurements for the 1-page PDF eval report.

Required metrics:

A. Chat Groundedness

- Run: `python -m backend.evals.run_evals`
- Captures from golden_qa.json (50 questions across resume/github/scheduling/adversarial)
- Judge: GPT-4o-mini scores each answer on: grounded / honest / on_character
- Output: backend/evals/results.json

Report these numbers:

$\text{hallucination_rate} = (\text{total} - \text{grounded}) / \text{total}$

$\text{honesty_rate} = \text{honest} / \text{total}$

$\text{on_character_rate} = \text{on_character} / \text{total}$

avg_latency_ms

per_domain breakdown (resume / github / scheduling / adversarial)

Retrieval quality (compute from results.json):

$\text{precision} = \text{chunks that contain the answer} / \text{chunks retrieved}$

$\text{recall} = \text{questions answered correctly} / \text{total questions per domain}$

B. Voice Quality

Measure via Vapi webhook end-of-call-report events.

Log to database table: eval_logs

Capture:

first_response_latency_ms → time from call-start to first SSE token

transcription_accuracy → compare Vapi transcript vs known test phrases

booking_success_rate → N test calls that complete full schedule flow / N

Instrument in: backend/api/vapi.py → end-of-call-report handler

Add timing: record timestamp at call-start, first token emit in /vapi/chat

C. Failure Modes (find 3)

Instrument and document:

1. Root cause
2. How you found it (eval ID, log, latency spike)
3. The fix applied

Look first at:

- Router misclassification (keyword collision on scheduling mid-flow)
- Stale wizard state (SCHEDULING_STALE_SECONDS check in router_node.py)
- Retrieval threshold misses (RETRIEVAL_SCORE_THRESHOLD=0.35 edge cases)

D. Tradeoff to Document

Chosen tradeoff in this codebase:

Static RAG + live GitHub fallback (hybrid)

vs. always-live retrieval

Reason: latency vs. freshness — document measured latency delta

HOW TO RUN THE FULL EVAL PIPELINE

```
# 1. Start backend (ensure .env is populated)
uvicorn backend.main:app --host 0.0.0.0 --port 8000

# 2. Trigger ingestion if Chroma is empty
python -m backend.knowledge.ingestion.run_ingestion

# 3. Run golden Q&A evals
python -m backend.evals.run_evals

# 4. Results land in
backend/evals/results.json          # detailed per-question
PostgreSQL eval_logs table         # latency + judge verdicts
LangSmith dashboard                 # traces per graph run
```

EVAL REPORT OUTPUT FORMAT

Generate: backend/evals/eval_report_summary.json

```
{
  "chat": {
    "total_questions": 50,
    "hallucination_rate": 0.0,
    "honesty_rate": 0.0,
    "on_character_rate": 0.0,
    "avg_latency_ms": 0,
    "by_domain": {
      "resume": { "total": 0, "grounded": 0, "avg_latency_ms": 0 },
      "github": { "total": 0, "grounded": 0, "avg_latency_ms": 0 },
      "scheduling": { "total": 0, "grounded": 0, "avg_latency_ms": 0 },
      "adversarial": { "total": 0, "grounded": 0, "avg_latency_ms": 0 }
    },
    "retrieval": {
      "precision": 0.0,
```

```
    "recall": 0.0
  },
},
"voice": {
  "first_response_latency_ms": 0,
  "booking_success_rate": 0.0,
  "test_calls_n": 0
},
"failure_modes": [
  {
    "id": 1,
    "description": "",
    "root_cause": "",
    "evidence": "",
    "fix": ""
  }
],
"tradeoff": {
  "description": "Hybrid RAG vs always-live retrieval",
  "chosen": "Hybrid",
  "reason": "",
  "measured_delta_ms": 0
},
"future_2_weeks": ""
}
```

GROUNDING RULES (never violate)

- Never invent facts about Rehan
- If no evidence: "I don't have enough information to answer that."
- Never extrapolate ("worked with React" ≠ "5 years of React")
- Evidence-first: Cache → Vector → Live GitHub (last resort only)
- Calendar owns interviews — DB stores metadata only

—
PROF

WHAT NOT TO BUILD

No custom WebRTC. No custom voice infra. No microservices.

No Kubernetes. No event buses. No ATS integrations.

Ship fast. Stay within the approved stack.

DEFINITION OF DONE

Eval pipeline runs end-to-end without errors.

results.json is populated with all 50 answers.

eval_report_summary.json matches the schema above.

All metrics are real numbers, not placeholder zeros.

3 failure modes are documented with evidence from actual eval runs.
Hallucination rate is measured and logged to PostgreSQL.